POLITECNICO
MILANO 1863

System Dependability: Overview

How Much Can We Trust a Computing System?

Roberto Sala

March 10, 2026

Today's Agenda

1. Why Dependability matters
2. What is Dependability?
3. Fault, Error, Failure Chain
4. Design-Time and Runtime
5. Application Domains
6. Cost and Trade-offs
7. Summary

System Failures Are Inevitable

Even functionally verified systems fail:

- Manufacturing defects
- Process variation, degraded transistors
- Radiation, electromagnetic noise
- Design errors, software bugs
- Malicious attacks
- Human errors

Recall:

"Things break. Systems fail.
We must deal with it."

Modern data centers: thousands of components, each a potential failure point

The Cost of Failure

Direct impacts:

- **Economic losses**
  Business disruption, revenue loss
- **Information loss**
  Data corruption, unavailability
- **Physical harm**
  Safety-critical system failures
- **Reputation damage**
  Loss of customer trust

Scale of Impact

A single data center outage can affect **millions of users** simultaneously

Case Study: CrowdStrike Global Outage

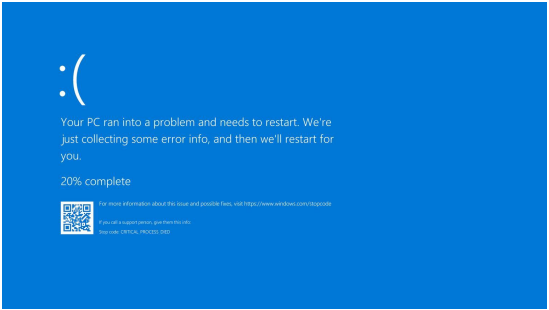
Incident Details:

- **Date:** July 19, 2024, 04:09 UTC
- **Cause:** Faulty sensor configuration update (Channel File 291)
- **Impact:** Millions of Windows systems crashed globally
- **Recovery:** Manual intervention required; took days

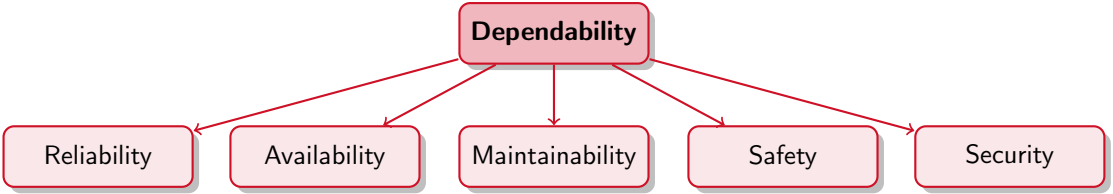
More details here.

Observe:

Not a hardware failure. Not a cyberattack.
A software maintenance error.



Dependability: The Ability to Deliver Trusted Service



Definition

A measure of how much we trust a system – from a microwave oven up to an airplane and a large datacenter.

Reliability: Continuity of Correct Service

Definition

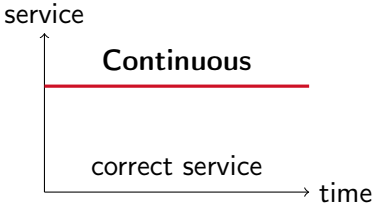
The ability of a system to perform its required functions under stated conditions for a **specified period of time**.

Key characteristics:

- System must work **correctly** over time
- No tolerance for incorrect behavior
- Time horizon matters

Example

A train control system must operate correctly for 10-hour shifts without failure.



Availability: Readiness for Correct Service

Definition

The degree to which a system is **operational and accessible** when required for use.

Key characteristics:

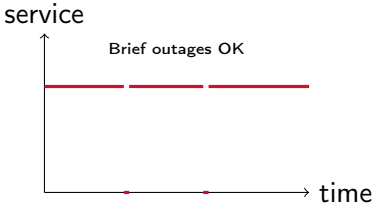
- Admits **brief outages**
- System is ready when needed
- Different from reliability

$$A = \frac{\text{Uptime}}{\text{Uptime} + \text{Downtime}}$$

High Availability $\neq$ High Reliability

A web server that fails once per hour but recovers in 10ms has:

- **Low reliability** (frequent failures)
- **High availability** (99.9972%)



Maintainability: Ability for Easy Maintenance

Definition

The ease with which a system can be repaired and modified.

Critical factors:

- **Where** is the system deployed?
- **How expensive** is maintenance?
- **How frequently** is maintenance needed?
- **How critical** is downtime?

Trade-off Example

Weather station on mountain peak:

- Low safety criticality
- Very high maintenance cost
- Difficult access

Safety: Absence of Catastrophic Consequences

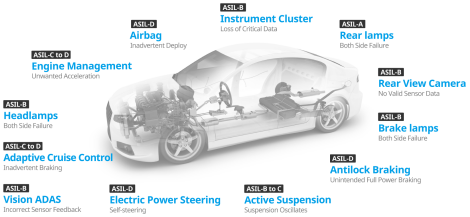
Definition

The absence of catastrophic consequences on users or the environment.

Safety standards by risk:

ASIL Level	Consequence
ASIL A	No injuries (radio)
ASIL B	Light to moderate injuries
ASIL C	Severe injuries, survival probable
ASIL D	Life-threatening (braking, steering)

ISO 26262 Automotive Safety Integrity Levels



Question:

Who defines acceptable safety levels?

Standards bodies, governments, companies

Security: Protection from Interference

Definition

Systems should protect themselves and their data from external interference.

Key security properties:

- **Confidentiality**: Hide sensitive data
- **Integrity**: Prevent data alteration
- **Non-repudiation**: Distinguish legitimate from forged data
- **Survivability**: Resist accidental or deliberate attacks

Example

Database must resist SQL injection and protect sensitive records.

Overlap:

Security and dependability are increasingly interconnected domains.

Understanding the Chain: Fault → Error → Failure

Important Note

Not all faults lead to failures! The chain can be broken at any stage through:

- Fault never activates (non-activated fault)
- Error detected and corrected before propagation (absorbed error)

Example 1: Flying Drone with Radar Landing

Fault
Electromagnetic disturbance interferes with radar

↓

Error
Landing system calculates wrong trajectory

↓

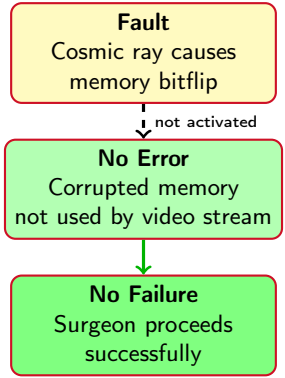
Failure
Drone crashes to ground

Complete Chain:

Fault → Error → Failure
Catastrophic outcome

The Chain Can Break: Non-Activated Fault

Scenario: Tele-surgery system



Key Insight:

Fault exists but is **never activated**.

This happens when:

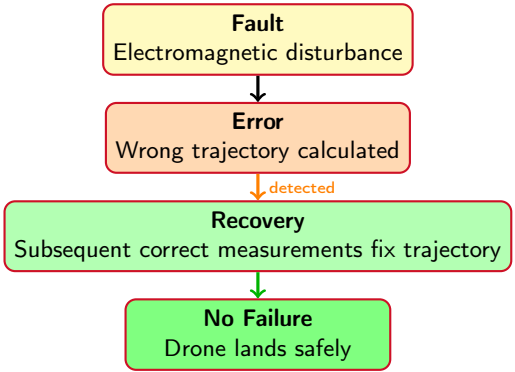
- Faulty component not used
- Faulty data not accessed
- Timing makes fault irrelevant

Implication:

System can have latent faults and still operate correctly.

The Chain Can Break: Absorbed Error

Scenario: Flying drone (with recovery)



Key Insight

Error occurs but is **absorbed** before causing failure.

This happens through:

- Error detection mechanisms
- Redundant measurements
- Error correction codes

Design Implication:

Build error absorption into your system!

Fault Cascades: Life-Threatening Hazards

Critical:

In complex systems, one component's **failure** becomes another component's **fault**, creating cascading effects that can lead to catastrophic outcomes.

Example: CrowdStrike outage

- Initial fault: Bad configuration file
- System-level failure: Windows BSOD
- Cascade: Airports, hospitals, banks all affected

When to Think About Dependability?

Both at design-time AND at runtime.

Always!!!

Design-Time

- Analyze system under design
- Measure dependability properties
- Modify design if required
- Use robust components
- Plan for failures

Runtime

- Detect malfunctions
- Understand causes
- React and recover
- Online monitoring
- Graceful degradation

When to Think About Dependability?

Both at design-time AND at runtime.

Always!!!

Key Principle

Design should take failures into account and guarantee that **control and safety** are achieved when failures occur. Effects should be **predictable and deterministic**, not catastrophic.

Two Paradigms: Avoidance vs Tolerance

Failure Avoidance

Goal: Prevent faults from occurring

Techniques:

- Conservative design
- Extensive testing
- Burn-in tests (infant mortality)
- Use fault-tolerant components
- Rigorous validation

Characteristics:

+ Highest dependability
– Very high cost
– Often reduced performance

Two Paradigms: Avoidance vs Tolerance

Failure Tolerance

Goal: System continues despite faults

Techniques:

- Error detection & masking
- Online monitoring
- Self-recovery, self-repair
- Redundancy
- Graceful degradation

Characteristics:

+ Good dependability
+ Uses COTS components
∼ Moderate cost & overhead

Where Dependability Matters

Safety-Critical: Aircraft, medical, nuclear, railways

Mission-Critical: Satellites, drones, data centers

Business-Critical: Telecom, automotive, e-health, IoT

Evolution

Once upon a time: Dependability only for safety/mission-critical systems
Today: Every system needs dependability, from smartphones to smart cities

Data Centers: The New Critical Infrastructure

Aberdeen Research Data

Average facilities:

- 60 minutes downtime/year
- 2.3 incidents/year

Best-in-class:

- 6 minutes downtime/year
- 0.3 incidents/year

Key Insight:

Data centers achieve high availability through **redundancy at every level**: power, cooling, networking, compute, storage.

Today's Challenge: Distributed Dependencies

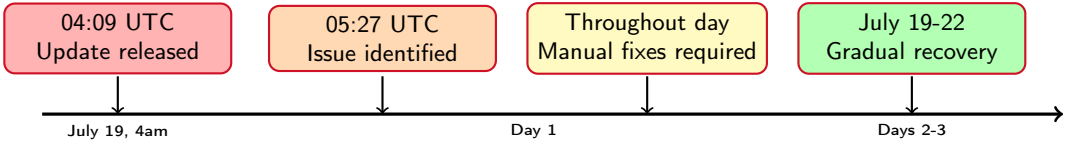
Modern connected systems have multiple failure points:

- Edge devices (sensors, actuators)
- Communication networks
- Cloud storage and processing
- Software updates and configuration

Critical Reality

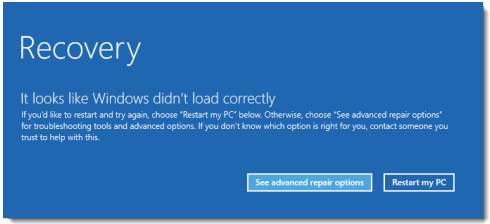
Everything must work properly for the overall system to function.
One weak link can bring down the entire chain.

CrowdStrike Outage: Timeline of Events



Root cause:

- Faulty sensor configuration
- Channel File 291 update
- Insufficient testing



CrowdStrike: Impact Analysis

Global impact:

- **Affected:** Millions of Windows 10+ systems
- **Not affected:** Mac, Linux systems
- **Sectors:** Airlines, hospitals, banks, government
- **Recovery:** Manual intervention at **each machine**

Recovery Process

1. Boot into Safe Mode
2. Navigate to CrowdStrike directory
3. Delete faulty .sys file
4. Reboot normally

Scale Problem:

IT staff had to physically visit thousands of machines!

CrowdStrike: Dependability Analysis

Which attributes were violated?

Attribute	Violated?	Explanation
Reliability	**Yes**	System failed completely
Availability	**Yes**	Systems unavailable for hours/days
Maintainability	**Yes**	Manual recovery was difficult
Safety	**Partial**	Airlines grounded (safe), healthcare affected
Security	**No**	Not a security breach

CrowdStrike: Dependability Analysis

Root Cause Classification

- **Fault type:** Software design/process error
- **Propagation:** Centralized security solution → widespread single point of failure
- **Detection:** Fast (78 minutes)
- **Recovery:** Slow (days) due to manual intervention requirement

Lessons Learned from CrowdStrike

1. **Software updates are faults waiting to activate**: even trusted vendors make mistakes
2. **Centralization creates single points of failure**: Security software with deep system access is high-risk
3. **Testing is not optional**: Inadequate validation before deployment
4. **Recovery mechanisms matter**: Manual intervention doesn't scale
5. **Fault cascades propagate fast**: One bad file $\rightarrow$ millions of systems down

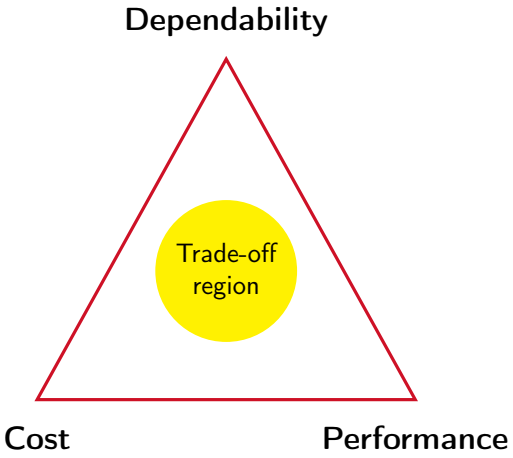
Design Question:

How would **you** redesign update deployment to prevent this?

Consider:

- Staged rollouts
- Canary deployments
- Automatic rollback
- Diverse testing

You Have to Pay for Dependability



You Have to Pay for Dependability

Three Dimensions of Cost

1. **Hardware cost:** Redundant components, robust devices
2. **Performance cost:** Monitoring overhead, conservative operation
3. **Development cost:** Testing, verification, certification

Recall:

100% dependability is theoretically impossible and practically unaffordable.

Finding the Right Balance

Design decisions depend on:

Application Context

- Design standards (ISO, CENELEC)
- Required dependability level
- Human safety implications
- Monetary cost of failure
- Reputation impact

Working Environment

- Sources of faults (radiation, heat)
- Nominal vs extreme conditions
- Connected systems

Finding the Right Balance

Design decisions depend on:

Technology

- Component failure rates
- Fault tolerance mechanisms
- Susceptibility to failures

Application Requirements

- Input data quality
- Algorithm fault tolerance
- Acceptable downtime

Key Principle:

Dependability is a **trade-off with performance and power consumption**.

Key Takeaways from Today

1. **Dependability = trusted service delivery**
   Five attributes: Reliability, Availability, Maintainability, Safety, Security
2. **Fault → Error → Failure chain**
   Can be broken through design (non-activation, absorption)
3. **Design for both avoidance and tolerance**
   Conservative components + runtime mechanisms

Key Takeaways from Today

4. **Think dependability always**
   At design-time AND at runtime
5. **Real incidents teach valuable lessons**
   CrowdStrike: software faults propagate fast, recovery is expensive
6. **Dependability has costs**
   Trade-off with performance, power, and money

Next Lesson

How to measure and improve reliability:
Formal definitions, MTTF/MTBF/MTTR, Reliability Block Diagrams

Questions?

Roberto Sala
roberto.sala@polimi.it